

Class $\rightarrow$ class className { classVarDec* subroutineDec* }

classVarDec $\rightarrow$ (static | field) type varName (, varName)* ;

type $\rightarrow$ int | char | boolean | className

SubroutineDec $\rightarrow$ (constructor | function | method) (Void | type) subroutineName
(parameterList) subroutineBody

parameterList $\rightarrow$ ((type varName) (, type varName)*)?

SubroutineBody $\rightarrow$ { VarDec* statements }

VarDec $\rightarrow$ Var type varName (, varName)* ;

className $\rightarrow$ identifier

subroutineName $\rightarrow$ identifier

varName $\rightarrow$ identifier

מקרא נחמד:	
X?	אם לא
X*	אם יותר
X Y	אם X או Y
(X Y)	X אחרי Y

Statements $\rightarrow$ Statement*

statement $\rightarrow$ letStatement | ifStatement | whileStatement | doStatement | returnStatement

letStatement $\rightarrow$ let VarName ([expression])? = expression ;

ifStatement $\rightarrow$ if (expression) { statements } (else { statements })?

whileStatement $\rightarrow$ while (expression) { statements }

do Statement $\rightarrow$ do SubroutineCall ;

return Statement $\rightarrow$ return expression? ;

expression $\rightarrow$ term (op term)*

term $\rightarrow$ integerConstant | StringConstant | keywordConstant | VarName |

VarName [expression] | SubroutineCall | (expression) | unaryOp term

SubroutineCall $\rightarrow$ SubroutineName (expressionList) | (className | VarName) . SubroutineName (expressionList)

expressionList $\rightarrow$ (expression (, expression)*)?

op $\rightarrow$ + | - | * | / | & | | | < | > | =

unaryOp $\rightarrow$ - | ~

keywordConstant $\rightarrow$ true | false | null | this